



Python Sets

Last Updated : 6 Apr, 2026

Python set is an unordered collection of multiple items having different datatypes. Sets are mutable, unindexed and do not contain duplicates. The order of elements in a set is not preserved and can change.

- Can store None values.
- Implemented using hash tables internally.
- Sets are not inherently thread-safe, synchronization is needed if used across threads.

Creating a Set

Set is created using curly braces {} with comma-separated values. It automatically stores only unique elements and does not maintain any specific order.

```
s = {1, 2, 3, 4}
print(s)
```



Output

```
{1, 2, 3, 4}
```

set() function

Sets can also be created using built-in [set\(\) function](#) with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'.

Note: A Python set cannot contain mutable objects like lists or dictionaries as elements because they are unhashable. However, mutable objects like lists can be used as input to create a set using the set() function.

```
s = set()
print(s)

s = set("GeeksForGeeks")
print(s)

# Creating a Set with the use of a List
s = set(["GFG", "For", "Geeks"])
print(s)

# Creating a Set with the use of a tuple
t = ("GFG", "for", "Geeks")
print(set(t))

# Creating a Set with the use of a dictionary
d = {"GFG": 1, "for": 2, "Geeks": 3}
```



```
print(set(d))
```

Output

```
set()
{'e', 'o', 'r', 'F', 'G', 'k', 's'}
{'Geeks', 'GFG', 'For'}
{'for', 'GFG', 'Geeks'}
{'for', 'GFG', 'Geeks'}
```

Unordered, Unindexed and Mutability

In set, the order of elements is not guaranteed to be the same as the order in which they were added. The output could vary each time we run the program. Also the duplicate items entered are removed by itself.

Sets do not support indexing. Trying to access an element by index (set[0]) raises a [TypeError](#).

```
s = {3, 1, 4, 1, 5, 9, 2}
```

```
print(s)
try:
    print(s[0])
except TypeError as e:
    print(e)
```

Output

```
{1, 2, 3, 4, 5, 9}
'set' object is not subscriptable
```

Adding Elements to a Set

We can add items to a set using [add\(\) method](#) and [update\(\) method](#). add() method can be used to add only a single item. To add multiple items we use update() method.

```
s = {1, 2, 3}
s.add(4)
s.update([5, 6])
print(s)
```

Output

```
{1, 2, 3, 4, 5, 6}
```

Accessing a Set

Because sets are unordered and unindexed, you cannot access elements using a specific index like set[0]. Instead, you must use a loop to iterate through the items or the in keyword to check for an item's existence.

```
s = {"Geeks", "For", "Geeks"}

for i in s:
    print(i, end=" ")
```

```
print("\n", "Geeks" in s)
```

Output

```
For Geeks
True
```

Explanation:

- for loop accesses each item in the set. Note that because sets are unordered, the items may appear in a different sequence every time the code is executed.
- Notice that although "Geeks" was added twice in the code, it only appears once in the output because sets automatically remove duplicates.
- "in" keyword efficiently checks if the string "Geeks" is present in s, returning True if found and False otherwise.

Removing Elements from the Set

1. Using remove() Method or discard() Method: remove() method removes a specified element from the set. If the element is not present in the set, it raises a KeyError.

discard() method also removes a specified element from the set. Unlike remove(), if the element is not found, it does not raise an error.

```
s = {1, 2, 3, 4, 5}
s.remove(3)
print(s)

try:
    s.remove(10)
except KeyError as e:
    print("Error:", e)

s.discard(4)
print(s)

s.discard(10)
print(s)
```

Output

```
{1, 2, 4, 5}
Error: 10
{1, 2, 5}
{1, 2, 5}
```

2. Using pop() Method: pop() method removes and returns an arbitrary element from the set. This means we don't know which element will be removed. If the set is empty, it raises a KeyError.

Note: If the set is unordered then there's no such way to determine which element is popped by using the pop() function.

```
s = {1, 2, 3, 4, 5}
val = s.pop()
print(val)
print(s)

s.clear()
```

```
try:
    s.pop()
except KeyError as e:
    print("Error:", e)
```

Output

```
1
{2, 3, 4, 5}
Error: 'pop from an empty set'
```

3. Using clear() Method: clear() method removes all elements from the set, leaving it empty.

```
s = {1, 2, 3, 4, 5}
s.clear()
print(s)
```

× ▶ 📄

Output

```
set()
```

Frozen Sets

A [frozenset](#) in Python is a built-in data type that is similar to a set but with one key difference that is immutability. This means that once a frozenset is created, we cannot modify its elements that is we cannot add, remove or change any items in it.

Like regular sets, a frozenset cannot contain duplicate elements. If no parameters are passed, it returns an empty frozenset.

```
fs = frozenset([1, 2, 3, 4, 5])
print(fs)

s = {3, 1, 4, 1, 5}
fs = frozenset(s)
print(fs)
```

× ▶ 📄

Output

```
frozenset({1, 2, 3, 4, 5})
frozenset({1, 3, 4, 5})
```

Typecasting Objects into Sets

Typecasting objects refers to converting various data types into a set. Python provides the set() constructor to perform this typecasting, allowing us to convert lists, tuples and strings into sets.

```
li = [1, 2, 3, 3, 4, 5, 5, 6, 2]
s = set(li)
print(s)

s = "GeeksforGeeks"
s = set(s)
print(s)
```

× ▶ 📄

```
d = {1: "One", 2: "Two", 3: "Three"}
s = set(d)
print(s)
```

Output

```
{1, 2, 3, 4, 5, 6}
{'G', 'k', 'e', 's', 'o', 'f', 'r'}
{1, 2, 3}
```

Recommended Problems: [Set Operations](#), [Set II](#)

Related Links:

- [Set Methods](#)
- [Maximum and Minimum in a Set](#)
- [Set operations](#)

Set in Python

[Visit Course](#)

Suggested Quiz

 10 Questions

Which of the following is the correct way to create a set in Python?

- ☐ A s={1,2,3}
- ☐ B s=[1,2,3]
- ☐ C s=(1,2,3)
- ☐ D s={1:"one",2:"two",3:"three"}

[Login to View Explanation](#)

1/10

< Previous Next >

Comment

S abhish...

136

Article Tags: [Misc](#) [Python](#) [python-set](#) [Python-Built-in-functions](#)



- Corporate & Communications Address:**
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)
- Registered Address:**
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company	Explore	Tutorials	Courses	Offline Centers	Preparation Corner
About Us	POTD	Programming	ML and Data	Noida	Interview
Legal	Practice	Languages	Science	Bengaluru	Corner
Privacy	Problems	DSA	DSA and	Hyderabad	Aptitude
Policy	Connect	Web	Placements	Kolkata	Puzzles
Careers	Blogs	Technology	Web		GfG 160
Contact Us	Upskill	AI, ML &	Development		System Design
Corporate	Courses	Data Science	Data Science		
Solution		DevOps	Programming		
Campus		CS Core	Languages		
Training		Subjects	DevOps &		
		GATE	Cloud		
		School	GATE		
		Subjects	Trending		
		Software and	Technologies		
		Tools			

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved